

SwiftPay Checkout for websites: collect payments from your own site into your SwiftPay account

Context

A business that sells on its own website (a shop, a SaaS, a booking site, a freelancer's portfolio) has no way to take payment into its SwiftPay account today. The invoice page needs an invoice drafted by hand per customer, the payment-request links need the payer to be signed in to SwiftPay, and there is no API, no webhook, no API key, and no reusable link a site can point at.

This plan gives those businesses three ways in, from no-code to full API, all landing on the same SwiftPay-hosted pay page and the same settlement path:

- 1 **Pay Links:** a reusable link or button (`/l/<slug>`) the business pastes into their site, email, or social bio. Fixed price or customer-chosen amount, optional product name and image, optional customer fields. No code.
- 2 **Checkout Sessions API:** the business's server creates a session with a secret API key, sends the customer to the returned URL, and gets a signed webhook when the payment lands. This is the Stripe Checkout shape developers already know.
- 3 ****Pay button and `pay.js`**:** a copy-paste button for a Pay Link, and a tiny script that opens the hosted checkout in a popup from any page, for sites that want a smoother flow without a backend.

Customers pay exactly as in the in-person plan: with a SwiftPay account, with any wallet on Arc, by card or bank through Circle Onramp, or with USDC bridged from another chain. The business is paid in USDC or EURC straight to its own wallet. SwiftPay never holds the money.

****This plan builds on `CHECKOUT-PLAN.md`**** (in-person checkout). It reuses that plan's `business_charges` table, the public pay page at `/c/<code>`, the on-chain verifier, the hash-less matcher for Onramp and bridge payments, and the expiry sweep. If this plan is built first, lift those pieces from Phase 1 and 2 of that document; nothing here replaces them.

SwiftPay is live on Arc mainnet. Everything below is additive and shipped behind the same `NEXT_PUBLIC_CHECKOUT_ENABLED` flag, with a testnet preview before a mainnet smoke test.

All paths are relative to `apps/web/` unless they start with `packages/`.

What exists and what it means for the design

- **Hosted, not embedded.** `next.config.mjs` sends `Content-Security-Policy: frame-ancestors 'none'` and `X-Frame-Options: DENY` on every route. An iframe checkout inside a merchant site would be blocked by the browser. The checkout is therefore a hosted page the customer is redirected to, or a popup opened by `pay.js`. If an embedded variant is wanted later, it needs a dedicated `/embed/*` route with its own `frame-ancestors` list of registered merchant domains.
- **Cross-site POST guard.** `proxy.ts` rejects any `/api/*` POST whose `Origin` is not SwiftPay's own (`isForgedCrossSiteRequest`). Server-to-server calls carry no `Origin` and pass. Browser calls from a merchant's site do not. The public API therefore lives under `/api/v1/` and that prefix is added to `crossSiteExemptApiPrefixes`; those routes authenticate with API keys, never with cookies, so the guard is not needed there.
- **Secrets and signing.** `lib/admin-auth.ts` already does constant-time secret comparison and minimum-length checks; `lib/wallet-session.ts` already signs with HMAC-SHA256. Both patterns are reused for API keys and webhook signatures. `lib/circle-webhook-signature.ts` is the inbound verifier for Circle; the outbound signer mirrors its header shape.
- **Email.** `lib/email/invoice-email.ts` sends through Resend (`RESEND_API_KEY`, `RESEND_EMAIL_DOMAIN`). Customer receipts reuse it.
- **Rate limiting** via `consumeRateLimit(key, max, windowSeconds)` in `lib/rate-limit.ts`.

- **Business identity:** `requireBusinessAccount` in `lib/account/auth.ts`, business profile in `business_account_profiles` (name, logo, website), keyed by the business wallet.
- **Settlement:** `claimTransferForCharge` (CHECKOUT-PLAN Phase 1) is the single place a payment is recorded. Webhook events are enqueued from there, so every payer path emits them.
- **Background work on Vercel** is cron-only today (`vercel.json`, daily jobs). Webhook delivery needs seconds, not hours: deliver inline at settlement with `after()` from `next/server` and let a cron retry failures.

Design summary

- **One charge, many doors.** A Pay Link visit, an API session, and a pay-button click all create a `business_charges` row (new kinds `HOSTED` and `PAY_LINK`) and send the customer to `/c/<code>`. The pay page grows a few optional parts: line items, merchant branding, customer fields, a "Return to <site>" button after payment, and a cancel link.
- **Keys.** Two key types per business, like every payments API: a **secret key** (`sk_live_...`, server only, creates sessions, reads sessions, manages nothing else) and a **publishable key** (`pk_live_...`, safe in a browser, can only start a session from a Pay Link). Only a SHA-256 hash of each key is stored; the full key is shown once at creation. Keys can be named, rotated, and revoked. Testnet deployments issue `sk_test_...` / `pk_test_...` so a key never works on the wrong network.
- **Webhooks.** Every settlement produces an event row with a stable id. Each enabled endpoint gets a delivery row. Deliveries are signed (`SwiftPay-Signature: t=<unix>,v1=<hex hmac> over <t>.<raw body>`), delivered immediately after settlement, retried on a backoff schedule by cron, and visible in the dashboard with status, response code, and a resend button. Event bodies carry the charge's `client_reference_id` and `metadata` so the merchant can match their own order.
- **Trust the chain, verify on return.** The success redirect carries `session_id` and a short-lived signed `return_token`. The merchant's site should never mark an order paid from the redirect alone: it either receives the webhook or calls `GET /api/v1/checkout/sessions/<id>` with its secret key. The docs say this plainly.
- **No custody, no refunds API.** Funds go wallet to wallet. A "Refund" action in the dashboard prefills `/send` to the payer's wallet with the charge code as memo, and the charge records the refund once the reverse transfer is seen on chain. There is no automatic refund.

Phase 1: API keys and Checkout Sessions

1a. Data

New `packages/database/supabase/business-web-checkout.sql` (same conventions as the other migrations: uuid keys, `wallet_address` references profiles, named checks, RLS on, `<table>_no_client_access`, grants only to `service_role`).

- `business_api_keys`: `id`, `wallet_address`, `name`, `kind` (`SECRET|PUBLISHABLE`), `network` (`MAINNET|TESTNET`), `prefix` (first 12 chars, for display), `key_hash` (sha256 hex, unique), `last_used_at`, `revoked_at`, `created_at`. Index (`wallet_address`, `created_at` desc).
- Extend `business_charges`: `kind` check gains `HOSTED` and `PAY_LINK`; add `api_key_id` uuid, `pay_link_id` uuid, `client_reference_id` text, `metadata` jsonb default '{}' (capped at 4 KB on write), `line_items` jsonb (array of {`name`, `quantity`, `unit_amount`}), `customer_email` text, `customer_name` text, `customer_fields` jsonb, `success_url` text, `cancel_url` text, `return_token_hash` text, `refund_tx_hash` text, `refunded_at` timestampz. Add status `REFUNDED` to the status check. Index (`wallet_address`, `client_reference_id`) for merchant lookups.

Modify `lib/account/db.ts`: add `apiKeys` to `accountTables`.

1b. Server library `lib/web-checkout/`

File	Purpose / what to reuse
keys.ts	generateApiKey(kind, network) → {key, prefix, hash} using crypto.randomBytes(24) base62 behind sk_live_, sk_test_, pk_live_, pk_test_; hashApiKey(key); authenticateApiKey(request, {kind}) reads Authorization: Bearer <key>, hashes, looks up the row (not revoked, right network via isArcMainnet()), touches last_used_at, returns {wallet_address, keyId, kind}. Constant-time compare on the hash, as in lib/admin-auth.ts.
sessions.ts	createCheckoutSession({wallet, keyId, body}): validates amount with validateChargeAmount (CHECKOUT-PLAN money-rules.ts), currency, success_url/cancel_url (https only, no credentials in URL, max 2 KB), metadata (flat object, 20 keys, 4 KB), line_items (max 50, total must equal amount), customer_email format; creates the business_charges row with kind: HOSTED and the API expiry (default 24h, max 7 days); mints return_token (random 32 bytes, hash stored). getCheckoutSession(wallet, id), listCheckoutSessions(wallet, {status, after}), expireCheckoutSession. serializeSession(charge) → the public API shape (id, url, `status: open`)
return.ts	buildSuccessRedirect(charge, token) appends session_id=<id>&return_token=<token> to success_url (preserving the merchant's own query), verifyReturnToken(charge, token).
errors.ts	webCheckoutErrors: invalidApiKey (401), wrongKeyKind (403), keyRevoked, sessionNotFound, invalidSession(msg) (400), rateLimited (429). JSON error shape for the public API: {error: {code, message}}.
http.ts	apiJson(data, status) with Cache-Control: no-store and CORS headers for /api/v1/* (Access-Control-Allow-Origin: * only on publishable-key routes; secret-key routes send none); handleOptions(); readApiBody(request) with a 64 KB cap; requireIdempotencyKey using readIdempotencyKey from lib/save/idempotency.ts and a (wallet_address, idempotency_key) unique column on charges (add in 1a).

1c. API routes (app/api/v1/)

All routes: export const runtime = "nodejs"; authenticate with authenticateApiKey; rate limit api:<keyId> 600/min and api-create:<wallet> 300/h; add /api/v1/ to crossSiteExemptApiPrefixes in proxy.ts.

- POST /api/v1/checkout/sessions (secret key) body {amount, currency?, description?, line_items?, customer?: {email?, name?}, collect?: ["email", "name", "phone"], success_url, cancel_url?, client_reference_id?, metadata?, expires_in?} → 201 {session} with url = <app>/c/<code>. Idempotency-Key header honoured.
- GET /api/v1/checkout/sessions/[id] (secret key) → {session}.
- GET /api/v1/checkout/sessions?status=&limit=&starting_after= (secret key) → {data, has_more}.
- POST /api/v1/checkout/sessions/[id]/expire (secret key) → cancels an OPEN session.
- GET /api/v1/me (either key) → {business: {name, username, wallet}, network, key: {kind, prefix}} for a quick "is my key working" check.

1d. Hosted pay page additions (components/checkout/pay-charge-page.tsx)

- Merchant header from business_account_profiles (logo, name, website link) and the session's description and line_items list above the amount.
- Customer fields block when collect is set or customer_email is missing and a receipt is wanted; written through POST /api/checkout/charges/[code]/customer (public, rate limited, OPEN only).
- After payment: "Return to <merchant site>" button → buildSuccessRedirect; while OPEN: a small "Cancel and go back" link → cancel_url when set.
- cancel_url and success_url are rendered as links only after new URL() parses them as https.

1e. Merchant dashboard: Developers page

- `app/business/developers/page.tsx` → `components/web-checkout/developers-hub.tsx`: API keys tab (create with name, copy once, revoke, last used), Sessions tab (list with status, amount, customer, `client_reference_id`, link to the pay page, "Expire" action).
- Routes under `app/api/business/developers/keys` (GET, POST, DELETE `[id]`) with `readActor + requireBusinessAccount`.
- Quick action "Developers" in `components/business/overview/business-quick-actions.tsx` behind the flag; nav entry via `platformNavItems` (`businessOnly: true`) and the `i18n` keys in `lib/i18n/index.ts` and every locale file.

Phase 2: Webhooks

2a. Data (same migration file, second section)

- `business_webhook_endpoints`: `id`, `wallet_address`, `url`, `description`, `secret_hash`, `secret_prefix`, `events text[]` (subset or `'{*}'`), `enabled boolean`, `network`, `created_at`, `disabled_at`, `failure_count integer default 0`. Secrets are `whsec_...`, shown once.
- `business_webhook_events`: `id (evt_...)`, `wallet_address`, `type`, `charge_id`, `payload jsonb`, `created_at`.
- `business_webhook_deliveries`: `id`, `event_id`, `endpoint_id`, `attempt integer`, `status (PENDING|SUCCEEDED|FAILED|EXHAUSTED)`, `response_status integer`, `response_excerpt text (1 KB)`, `next_attempt_at timestampz`, `delivered_at`, `created_at`. Index on `(status, next_attempt_at)`.

2b. Library `lib/web-checkout/webhooks.ts`

- `emitChargeEvent(type, charge)` called from `claimTransferForCharge` (paid and partial), `expireStaleCharges` (expired), `cancelCharge`, and the refund recorder. Types: `checkout.session.completed`, `checkout.session.partially_paid`, `checkout.session.expired`, `checkout.session.cancelled`, `checkout.session.refunded`. Payload is `{id, type, created, data: {object: serializeSession(charge)}}`.
- For each enabled endpoint subscribed to the type: insert a PENDING delivery, then attempt it inside `after()` so the response to the payer is not delayed.
- `deliver(deliveryId)`: build the body, sign with the endpoint secret (SwiftPay-Signature: `t=<unix>,v1=<hmac_sha256(secret, "<t>.<body>")>`), add SwiftPay-Event-Id and SwiftPay-Delivery-Attempt, POST with a 10 s timeout, follow no redirects, read at most 64 KB. 2xx → SUCCEEDED. Otherwise schedule the next attempt: 1 min, 5 min, 30 min, 2 h, 12 h, 24 h, then EXHAUSTED and `failure_count` incremented; an endpoint with 50 consecutive failures is auto-disabled and the business is notified in the bell.
- SSRF guard before every call: https only, hostname resolves to a public address (reject loopback, link-local, private ranges, metadata addresses), port 443 or 8443, no credentials in the URL.
- `verifyWebhookSignature(secret, header, body, {tolerance: 300})` exported for the SDK and shown in the docs; the same code is used by the "Send test event" button.

2c. Routes and cron

- `app/api/business/developers/webhooks` (GET, POST), `[id]` (PATCH enable/disable/events, DELETE), `[id]/test` (sends a ping event), `deliveries?endpoint=` (list), `deliveries/[id]/resend`.
- `app/api/cron/webhooks/route.ts` (`isCronAuthorized`, `maxDuration = 60`): picks PENDING deliveries with `next_attempt_at <= now()` in batches of 100 and runs `deliver`. Add to `vercel.json`; if the Vercel plan allows, schedule it every 5 minutes, otherwise hourly, and say in the docs that retries after the first inline attempt follow that cadence.

2d. Dashboard

Webhooks tab in the Developers hub: endpoint list with status, recent deliveries with response code and body excerpt, resend, test event, and the signing secret shown once with a "Rotate" action.

Phase 3: Pay Links, pay button, `pay.js`

3a. Data

- `business_pay_links`: `id`, `wallet_address`, `slug` (unique, 6 to 40 chars, lowercase, chosen or generated), `title`, `description`, `image_url`, `currency`, `amount` (null = customer chooses), `min_amount`, `max_amount`, `suggested_amounts` `text[]`, `allow_tip` boolean, `quantity_enabled` boolean, `max_quantity`, `collect` `text[]` (`email`, `name`, `phone`, `address`, `note`), `success_url`, `success_message`, `active` boolean, `uses_limit` integer, `uses_count` integer, `expires_at`, `created_at`, `updated_at`.

3b. Public pages and routes

- `/l/[slug]` (`app/l/[slug]/page.tsx` → `components/web-checkout/pay-link-page.tsx`): product card (image, title, description), amount or amount picker, quantity, tip, customer fields, Continue → creates a `PAY_LINK` charge via `POST /api/checkout/pay-links/[slug]/sessions` (public, rate limited per IP and per link, respects `active`, `uses_limit`, `expires_at`) → `/c/<code>`. Keep `/l` out of the proxy's protected matchers.
- `GET /api/checkout/pay-links/[slug]` (public) → link details for the page and for `pay.js`.
- `POST /api/v1/pay-links/[slug]/sessions` (publishable key, CORS *): the same creation for sites that build their own button; returns `{url}` only.

3c. Merchant dashboard

`app/business/pay-links/page.tsx` → list, create/edit form, activate/deactivate, copy link, and a "Get button" panel that outputs three snippets: a plain `<a>` button, the `pay.js` popup button, and a QR (reusing `LazyQRCodeSVG` and the poster helper from `CHECKOUT-PLAN`).

3d. `pay.js`

- `public/pay.js` (static, versioned path `public/pay/v1.js`): finds `[data-swiftpay-link]` and `[data-swiftpay-session-url]` elements, on click opens the hosted page in a centered popup (`window.open`, 480×760) and falls back to a full redirect when the popup is blocked; listens for a `postMessage` from the hosted page (`{type: "swiftpay:paid", session_id}`) sent only to the opener's origin, then calls the merchant's `data-swiftpay-onpaid` callback or redirects to `success_url`. It sets `rel="noopener"` where it redirects and never reads cookies.
- The hosted page posts that message only when `window.opener` exists and the charge is PAID, with the opener origin taken from the session's `success_url` origin.

Phase 4: Developer experience and finishing

- **Docs page** `app/developers/page.tsx` (public): quick start (create key, create session with `curl`, redirect, handle webhook), API reference generated from a small typed table in `lib/web-checkout/docs.ts`, webhook verification snippets in Node and Python, Pay Link and button guide, testnet instructions. Linked from the Developers hub and the landing page footer.
- **Node SDK** `packages/sdk-node` (@swiftpay/node): `new SwiftPay(secretKey), checkout.sessions.create/retrieve/list/expire, webhooks.constructEvent(body, header, secret)`. Zero dependencies, built with `tsc`, published later; for now consumed from the monorepo in docs.
- **Email receipt**: `lib/email/receipt-email.ts` sending through the same Resend helper when `customer_email` is present: amount, business name, tx explorer link, "Return to <site>" link.
- **Payments list and export**: `app/business/payments/page.tsx` listing charges of every kind with filters (kind, status, date) and a CSV export (`GET /api/business/payments/export.csv`).

- **Refund assist:** "Refund" on a PAID charge →
`/send?to=<payer>&amount=<amt>&token=<cur>&memo=<code>&refund=<code>`; the dashboard records the reverse transfer through the existing receipt verifier (destination = payer wallet) and sets `REFUNDED`, emitting the webhook.
- **Overview card:** online revenue today and this month, open sessions, failed webhooks, using `chargeSummary` from CHECKOUT-PLAN extended with a `kind` filter.
- `.env.example` (root and `apps/web`): nothing new required; document that webhooks and the cron use `CRON_SECRET`, receipts use `RESEND_*`.

Security

- Secret keys are never stored or logged in clear; only the hash and a display prefix. Responses never echo a full key except at creation. Keys are bound to a network and refused on the other.
- Publishable keys can do exactly one thing: start a session from an active Pay Link. They cannot read sessions or set amounts outside the link's own rules.
- `success_url` and `cancel_url` must be https and under 2 KB; the pay page only ever navigates to URLs stored server-side at session creation, never to a URL from the query string.
- `return_token` proves nothing to the merchant beyond "this redirect came from SwiftPay"; the docs and the API response say that payment state comes from the webhook or the GET endpoint.
- Webhook delivery has an SSRF guard, a timeout, a size cap, and no redirect following. Signatures use a per-endpoint secret; timestamps are checked against replay with a 5 minute tolerance.
- Public creation endpoints (Pay Link sessions, customer fields) are rate limited per IP, per link, and per business, and open charges per business are capped (for example 500) so a bot cannot flood the table.
- `metadata` and `customer_fields` are stored as given but rendered escaped; they never reach another customer's page.

Rollout on the live app

- 1 Run `business-web-checkout.sql` (additive: new tables and new nullable columns on `business_charges`).
- 2 Deploy behind `NEXT_PUBLIC_CHECKOUT_ENABLED`; the `/api/v1/` routes are reachable only with a key, and no business has a key until the Developers page is visible.
- 3 Testnet preview: run the full verification below with `sk_test_` keys against a throwaway site (a one-file Express or Next app in `packages/examples/web-checkout-demo`, which doubles as the docs example).
- 4 Mainnet: one real business creates a `sk_live_` key, creates a session for about 0.50 USDC, pays it by wallet, confirms the webhook delivery and the GET response, then the flag goes on.
- 5 Watch `business_webhook_deliveries` for EXHAUSTED rows and `business_api_keys.last_used_at` for adoption in the first weeks.

Verification

Unit tests (`node:test`, `.test.mjs`, under `lib/web-checkout/__tests__`, chained into `test:web-checkout` in `apps/web/package.json`):

- `keys.test.mjs`: prefix per kind and network, hash lookup, revoked and wrong-network keys refused.
- `sessions.test.mjs`: body validation (amount bounds, https URLs, metadata caps, line items sum), `serializeSession` shape and status mapping, idempotency replay returns the same session.
- `return.test.mjs`: success redirect keeps the merchant's query string; token verify and expiry.
- `webhooks.test.mjs`: signature header format and verification (good, tampered body, stale timestamp), retry schedule, SSRF guard rejects private hosts and http URLs.
- `pay-links.test.mjs`: slug rules, amount rules (fixed, range, suggested), uses limit, inactive link.

Commands:

```
cd apps/web && pnpm run typecheck && pnpm run test:web-checkout && pnpm run test:checkout
```

Manual end-to-end (testnet preview, then the mainnet smoke test from Rollout):

- 1 Business → Developers → create a secret key; GET `/api/v1/me` with it returns the business.
- 2 From the demo site, create a session (curl and the Node SDK), open url, pay with a wallet, land back on the demo's success page with `session_id`; the demo's webhook handler logs `checkout.session.completed` with a valid signature; GET the session shows `paid` and the tx hash.
- 3 Repeat the payment with a SwiftPay account (signed-out, then sign-in redirect), with card/bank on the Onramp sandbox, and with a Base Sepolia bridge; each produces one `completed` event.
- 4 Create a Pay Link with a fixed price and one with a customer-chosen amount; pay both from the public link and from the `pay.js` popup button on the demo site; confirm the popup closes and the callback fires.
- 5 Webhooks: point an endpoint at a URL that returns 500, watch the delivery go PENDING → retry → EXHAUSTED on the schedule, resend it manually after fixing the URL; "Send test event" works; an endpoint set to `http://` or a private IP is refused at creation.
- 6 Negatives: revoked key → 401; publishable key on a secret route → 403; `success_url` with `javascript:` or `http://` → 400; metadata over 4 KB → 400; a second session with the same `Idempotency-Key` returns the first; an expired session shows the cancel link and emits `expired`.
- 7 Refund assist: refund a paid charge from the dashboard, confirm `REFUNDED` and the webhook.
- 8 Email receipt arrives when a customer email was collected; none is sent otherwise.

Open risks

- **Webhook timeliness on Vercel.** The inline attempt in `after()` covers the normal case; retries depend on the cron cadence the plan allows. Say so in the docs and expose the delivery log.
- **Popup blockers.** `pay.js` falls back to a redirect, which loses the merchant's in-page state; the redirect flow is the documented default, the popup is an enhancement.
- **KYB and compliance.** A business taking online payments from strangers may need verification beyond today's optional business verification. Gate `sk_live_` key creation on `verification_status = VERIFIED` if the business team decides so; the plan leaves the switch in `keys.ts`.
- **Customer support load.** Guests paying by card go through Circle's identity check; failed checks land as expired sessions. The merchant sees them in the Sessions tab, with the Onramp session id, so support can follow up.
- **Price in USDC only.** Merchants pricing in local currency need a displayed conversion; the pay page can show an approximate local amount using `lib/use-conversion-rates.ts`, but the charge is always a fixed USDC or EURC amount.